

P3827R0

Wording for US NB comment 9

Published Proposal, 2025-09-01

Author:

[Arthur O'Dwyer](#)

Audience:

WG21

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

US NB comment 9 objects to the new notion of "replaceability," which is unused anywhere in the Standard and whose precise semantics are unclear even to library implementors.

Table of Contents

1 US NB comment 9

2 Important note

3 Proposed wording

3.1 [basic.types.general]

3.2 [class.pre]

3.3 [class.prop]

3.4 [diff.cpp23.dcl.dcl]

3.5 [diff.cpp23.library]

3.6 [expr.prim.lambda.closure]

3.7 [lex.name]

3.8 [library.class.props]

- 3.9 [meta.reflection.traits]
- 3.10 [meta.syn]
- 3.11 [meta.type.synop]
- 3.12 [meta.unary.prop]

References

Informative References

§ 1. US NB comment 9

The new notion of "replaceable types" (6.9.1) is confusingly similar to the existing notions of "transparently replaceable objects" (6.8.4) and "replaceable functions", such as replaceable allocation functions (9.6.5) and replaceable contract-violation handlers (6.11.3).

The new notion of "replaceable" is not used by any C++26 library machinery. Therefore it does not need to exist in C++26. The proposed change is entirely removal of various terms of art all of which are dangling loose ends. The removal of this type trait has no effect on the rest of C++.

There are 88 instances of the string "[replaceab](#)" in the Committee Draft. Adopting the proposed change will remove 36 instances concerned with [replaceable_if_eligible](#), leaving:

- 13 instances concerned with "system_context_replaceability"
- 4 concerned with "transparently replaceable"
- 35 instances concerned with replaceable functions

§ 2. Important note

We needn't touch anything not listed below. Literally nothing in the Standard depends on this inutile notion.

§ 3. Proposed wording

The wording in this section is relative to the Committee Draft of C++26, [\[N5013\]](#).

§ 3.1. [basic.types.general]

Modify [basic.types.general] as follows:

9. Arithmetic types ([basic.fundamental]), enumeration types, pointer types, pointer-to-member types ([basic.compound]), `std::nullptr_t`, and cv-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, trivially relocatable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially relocatable types*. ~~Cv-qualified scalar types, replaceable class types ([class.prop]), and arrays of such types are collectively called *replaceable types*.~~ Scalar types, standard-layout class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types ([class.prop]), array types, and cv-qualified versions of these types are collectively called *implicit-lifetime types*.
10. A type is a *literal type* if it is: [...]

§ 3.2. [class.pre]

Modify [class.pre] as follows:

class-property-specifier:

```
final  
trivially_relocatable_if_eligible  
replaceable_if_eligible
```

[...]

5. Each *class-property-specifier* shall appear at most once within a single *class-property-specifier-seq*. Whenever a *class-key* is followed by a *class-head-name*, the identifier `final`, `trivially_relocatable_if_eligible`, or ~~`replaceable_if_eligible`~~, and a colon or left brace, the identifier is interpreted as a *class-property-specifier*.

[Example:

```
struct A;  
struct A final {};           // OK, definition of struct A,  
                             // not value-initialization of variable final  
  
struct X {  
    struct C { constexpr operator int() { return 5; } };  
    struct B trivially_relocatable_if_eligible : C{};  
                             // OK, definition of nested class B,  
                             // not declaration of a bit-field member  
                             // trivially_relocatable_if_eligible  
};
```

—end example]

§ 3.3. [class.prop]

Modify [class.prop] as follows:

[...]

4. A class *C* is a *trivially relocatable class* if it is eligible for trivial relocation and

- has the `trivially_relocatable_if_eligible` *class-property-specifier*,
- is a union with no user-declared special member functions, or
- is default-movable.

5. [Note: A class with const-qualified or reference non-static data members can be trivially relocatable. —end note]

6. A class *C* is *eligible for replacement* unless

- it has a base class that is not a replaceable class,
- it has a non-static data member that is not of a replaceable type,
- overload resolution fails or selects a deleted constructor when direct-initializing an object of type *C* from an xvalue of type *C* ({`del.init.general`}),
- overload resolution fails or selects a deleted assignment operator function when assigning to an lvalue of type *C* from an xvalue of type *C* ({`expr.assign`}, {`over.assign`}), or
- it has a deleted destructor.

7. A class *C* is a *replaceable class* if it is eligible for replacement and

- has the `replaceable_if_eligible` *class-property-specifier*,
- is a union with no user-declared special member functions, or
- is default-movable.

8. [Note: Accessibility of the special member functions is not considered when establishing trivial relocatability or replaceability. —end note]

9. [Note: Not all trivially copyable classes are trivially relocatable or replaceable. —end note]

10. A class *S* is a *standard-layout class* if it: [...]

§ 3.4. [diff.cpp23.dcl.dcl]

Modify [diff.cpp23.dcl.dcl] as follows:

1. Affected subclause: [dcl.decl.general]

Change: Introduction of `trivially_relocatable_if_eligible` and ~~`replaceable_if_eligible`~~ as ~~identifiers~~ an identifier with special meaning.

Rationale: Support declaration of trivially relocatable ~~and replaceable~~ types.

Effect on original feature: Valid C++ 2023 code can become ill-formed.

[Example:

```
struct C {};  
struct C replaceable_if_eligible trivially_relocatable_if_eligible {};    // was well-f  
                                         // now ill-formed (redefines C)
```

—end example]

§ 3.5. [diff.cpp23.library]

NOTE: This clause should point to [cpp.replace.general], not [res.on.macro.definitions]; that is the subject of US NB comment 57.

Modify [diff.cpp23.library] as follows:

2. Affected subclause: [res.on.macro.definitions]

Change: Additional restrictions on macro names.

Rationale: Avoid hard to diagnose or non-portable constructs.

Effect on original feature: Names of special identifiers may not be used as macro names. Valid C++ 2023 code that defines ~~`replaceable_if_eligible` or `trivially_relocatable_if_eligible` as macros~~ a macro named `trivially_relocatable_if_eligible` is invalid in this revision of C++.

§ 3.6. [expr.prim.lambda.closure]

Modify [expr.prim.lambda.closure] as follows:

3. The closure type is not an aggregate type ([`dcl.init.aggr`]); it is a structural type ([`temp.param`]) if and only if the lambda has no *lambda-capture*. An implementation may define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:

- the size and/or alignment of the closure type,
- whether the closure type is trivially copyable ([`class.prop`]),
- whether the closure type is trivially relocatable ([`class.prop`]),
- ~~whether the closure type is replaceable ([`class.prop`])~~, or
- whether the closure type is a standard-layout class ([`class.prop`]).

An implementation shall not add members of rvalue reference type to the closure type.

§ 3.7. [`lex.name`]

Modify [`lex.name`] as follows:

2. The identifiers in Table 4 have a special meaning when appearing in a certain context. When referred to in the grammar, these identifiers are used explicitly rather than using the *identifier* grammar production. Unless otherwise specified, any ambiguity as to whether a given *identifier* has a special meaning is resolved to interpret the token as a regular *identifier*.

Table 4 — Identifiers with special meaning

<code>final</code>	<code>import</code>	<code>post</code>	<code>replaceable_if_eligible</code>
<code>override</code>	<code>module</code>	<code>pre</code>	<code>trivially_relocatable_if_eligible</code>

§ 3.8. [`library.class.props`]

Modify [`library.class.props`] as follows:

1. Unless explicitly stated otherwise, it is unspecified whether any class described in [support] through [exec] and [depr] is a trivially copyable class, a standard-layout class, or an implicit-lifetime class ([class.prop]).
2. Unless explicitly stated otherwise, it is unspecified whether any class for which trivial relocation (i.e., the effects of `trivially_relocate` ([obj.lifetime])) would be semantically equivalent to move-construction of the destination object followed by destruction of the source object is a trivially relocatable class ([class.prop]).
- ~~3. Unless explicitly stated otherwise, it is unspecified whether a class `C` is a replaceable class ([class.prop]) if assigning an xvalue `a` of type `C` to an object `b` of type `C` is semantically equivalent to destroying `b` and then constructing from `a` in `b`'s place.~~

§ 3.9. [meta.reflection.traits]

Modify [meta.reflection.traits] as follows:

```
[...]  
constexpr bool is_trivially_copyable_type(info type);  
constexpr bool is_trivially_relocatable_type(info type);  
constexpr bool is_replaceable_type(info type);  
constexpr bool is_standard_layout_type(info type);  
[...]
```

§ 3.10. [meta.syn]

Modify [meta.syn] as follows:

```
[...]  
constexpr bool is_trivially_copyable_type(info type);  
constexpr bool is_trivially_relocatable_type(info type);  
constexpr bool is_replaceable_type(info type);  
constexpr bool is_standard_layout_type(info type);  
[...]
```

§ 3.11. [meta.type.synop]

Modify [meta.type.synop] as follows:


```

[...]  

template<class T> struct is_trivially_copyable;  

template<class T> struct is_trivially_relocatable;  

template<class T> struct is_replaceable;  

template<class T> struct is_standard_layout;  

[...]  

template<class T>  

    constexpr bool is_implicit_lifetime_v = is_implicit_lifetime<T>::value;  

template<class T>  

    constexpr bool is_replaceable_v = is_replaceable<T>::value;  

template<class T>  

    constexpr bool has_virtual_destructor_v = has_virtual_destructor<T>::value;

```

§ 3.12. [meta.unary.prop]

Modify Table 54 in [meta.unary.prop] as follows:

Template	Condition	Preconditions
template<class T> struct is_trivially_copyable;	T is a trivially copyable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or <i>cv</i> void.
template<class T> struct is_trivially_relocatable;	T is a trivially relocatable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or <i>cv</i> void.
template<class T> struct is_replaceable;	T is a replaceable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or <i>cv</i> void.
template<class T> struct is_standard_layout;	T is a standard-layout type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or <i>cv</i> void.

§ References

§ Informative References

[N5013]

Committee Draft — Programming Languages — C++. August 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5013.pdf>